Essentials of AI

Python

- Python is a high-level, interpreted, and user-friendly programming language. It is widely used in fields like web development, artificial intelligence, data science, automation, and more.
- Easy to learn – The syntax is simple and close to English.
- Powerful – Used by companies like Google, YouTube, NASA, and Instagram.
- Versatile – Can be used to build websites, apps, games, and analyze data.

Feature	Description
Simple Syntax	Easy to read and write.
Interpreted	Executes line by line.
Free & Open Source	Anyone can use or modify.
Portable	Same code works on Windows, Mac, Linux.
Object-Oriented	Based on objects and classes.
Large Library	Ready-made modules for almost anything.

How Python Works

- You write Python code in a file (example: program.py).
- The Python interpreter reads the code line by line and executes it.

- Basic Concepts of Python Programming

Python as an Interpreted Language

- Python executes the code line by line.
- It does not require compilation (like C or Java).
- This makes debugging easier for beginners.

Writing and Running Python Code

- You can write Python code in:
- IDLE (comes with Python installation)
- Online compilers (like Replit, Jupyter Notebook, Google Colab)
- Text editors (like Visual Studio Code, PyCharm)

```
print("Hello, World!")
```

OUTPUT- Hello, World!

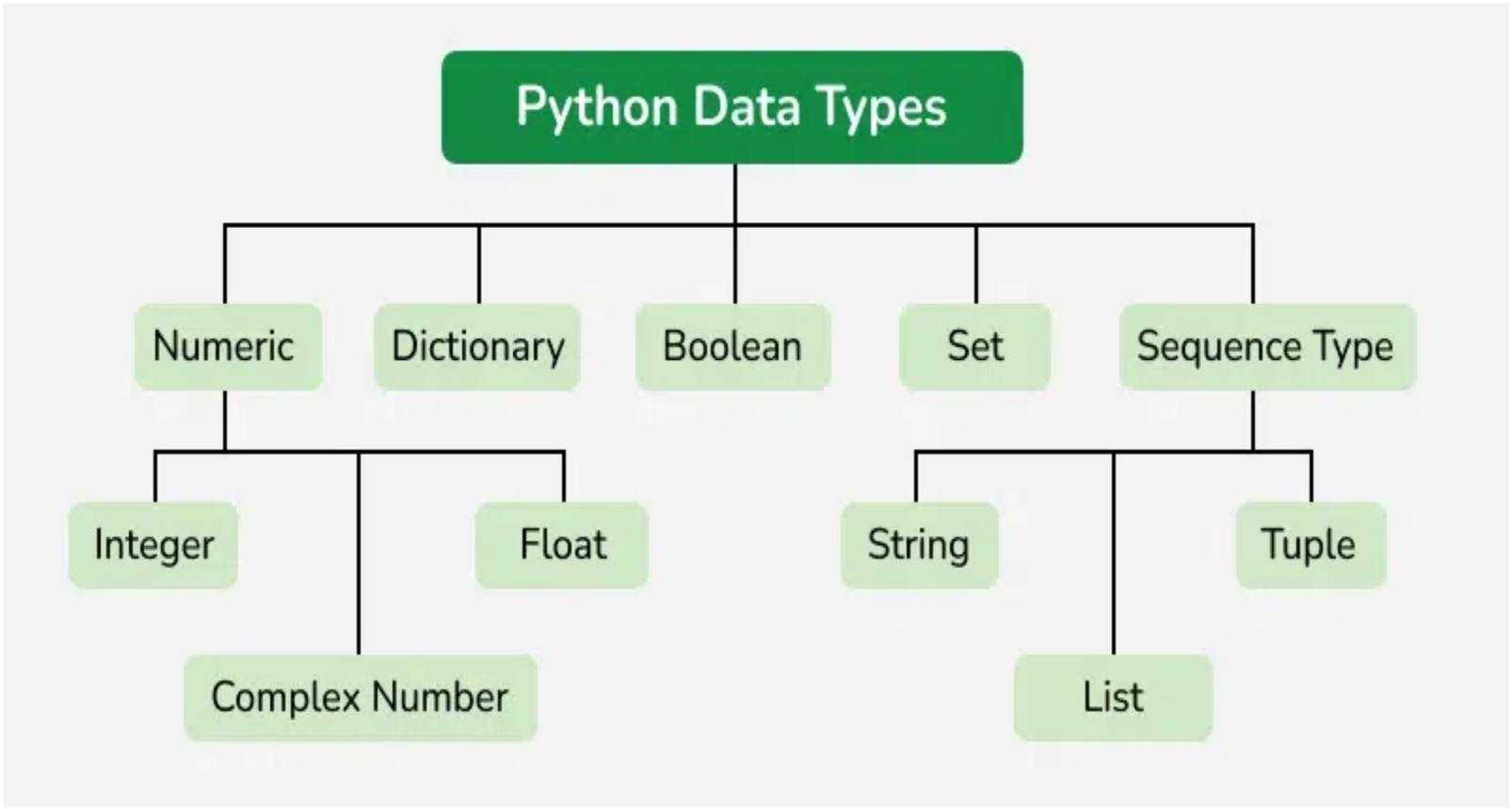
This program prints text on the screen using the print() function.

Python is:

- Easy to read
- Beginner-friendly
- Powerful and versatile

Data types in Python

- Data types in Python are a way to classify data items. They represent the kind of value, which determines what operations can be performed on that data. Since everything is an object in Python programming, Python data types are classes and variables are instances (objects) of these classes.
- The following are standard or built-in data types in Python:
- Numeric: int, float, complex
- Sequence Type: string, list, tuple
- Mapping Type: dict
- Boolean: bool
- Set Type: set, frozenset
- Binary Types: bytes, bytearray, memoryview


Python Data Types
Numeric
Dictionary
Boolean
Set
Sequence Type
Integer
Float
Complex Number
String
Tuple
List

- x = 50 # int
- x = 60.5 # float
- x = "Hello World" # string
- x = ["geeks", "for", "geeks"] # list
- x = ("geeks", "for", "geeks") # tuple

1. Numeric Data Types

A numeric value can be an integer, a floating number or even a complex number. These values are defined as int, float and complex classes.

- Integers: value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). There is no limit to how long an integer value can be.
- Float: value is represented by float class. It is a real number with a floating-point representation. It is specified by a decimal point.
- Complex Numbers: It is represented by a complex class. It is specified as (real part) + (imaginary part)j. For example - 2+3j

```
a = 5
print(type(a))

b = 5.0
print(type(b))

c = 2 + 4j
print(type(c))
```

2. Sequence Data Types

A sequence is an ordered collection of items, which can be of similar or different data types. Sequences allow storing of multiple values in an organized and efficient fashion.

String Data Type

- Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class.
- Strings in Python can be created using single quotes, double quotes or even triple quotes. We can access individual characters of a String using index.

```
s = 'Welcome to the Geeks World'
print(s)

# check data type
```

- print(type(s))

```
# access string with index
```

- print(s[1])
- print(s[2])
- print(s[-1])

List Data Type

- Lists are similar to arrays found in other languages. They are an ordered and mutable collection of items. It is very flexible as items in a list do not need to be of the same type.
- Creating a List in Python
- Lists in Python can be created by just placing sequence inside the square brackets[].

```
# Empty list
a = []

# list with int values
a = [1, 2, 3]
print(a)

# list with mixed values int and String
b = ["Geeks", "For", "Geeks", 4, 5]
print(b)
```

Tuple Data Type

- Tuple is an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable. Tuples cannot be modified after it is created.
- Creating a Tuple in Python

In Python, tuples are created by placing a sequence of values separated by a ‘comma’ with or without the use of parentheses for grouping data sequence. Tuples can contain any number of elements and of any datatype (like strings, integers, lists, etc.).

```
# initiate empty tuple
tup1 = ()

tup2 = ('Geeks', 'For')
print("\nTuple with the use of String: ", tup2)
```

```
tup1 = (1, 2, 3, 4, 5)

# access tuple items
print(tup1[0])
print(tup1[-1])
print(tup1[-3])
```

3. Boolean Data Type

- Boolean Data type is one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true) and those equal to False are falsy (false). However non-Boolean objects can be evaluated in a Boolean context as well and determined to be true or false. It is denoted by class bool.

- print(type(True))
- print(type(False))
- print(type(true))

- Truthy and Falsy Values
- In Python, truthy and falsy values are values that evaluate to True or False in a Boolean context. Truthy values behave like True, while falsy values behave like False when used in conditions.

```
if 1:
  print("1 is truthy")


if not 0:
    print("0 is falsy")
```

4. Set Data Type

- In Python Data Types, Set is an unordered collection of data types that is iterable, mutable, and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

- # initializing empty set
- s1 = set()

- s1 = set("GeeksForGeeks")
- print("Set with the use of String: ", s1)

- s2 = set(["Geeks", "For", "Geeks"])
- print("Set with the use of List: ", s2)

Access Set Items

- Set items cannot be accessed by referring to an index, since sets are unordered the items have no index. But we can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the keyword in.

```
set1 = set(["Geeks", "For", "Geeks"]) #Duplicates are removed automatically
print(set1)

# loop through set
for i in set1:
  print(i, end=" ") #prints elements one by one

# check if item exist in set
print("Geeks" in set1)
```

5. Dictionary Data Type

- It is a collection of data values, used to store data values like a map, unlike other Python Data Types, a Dictionary holds a key: value pair. Key-value is provided in dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon : , whereas each key is separated by a ‘comma’.

```
# initialize empty dictionary
d = {}

d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d)

# creating dictionary using dict() constructor
d1 = dict({1: 'Geeks', 2: 'For', 3: 'Geeks'})
print(d1)
```

Accessing Key-value in Dictionary

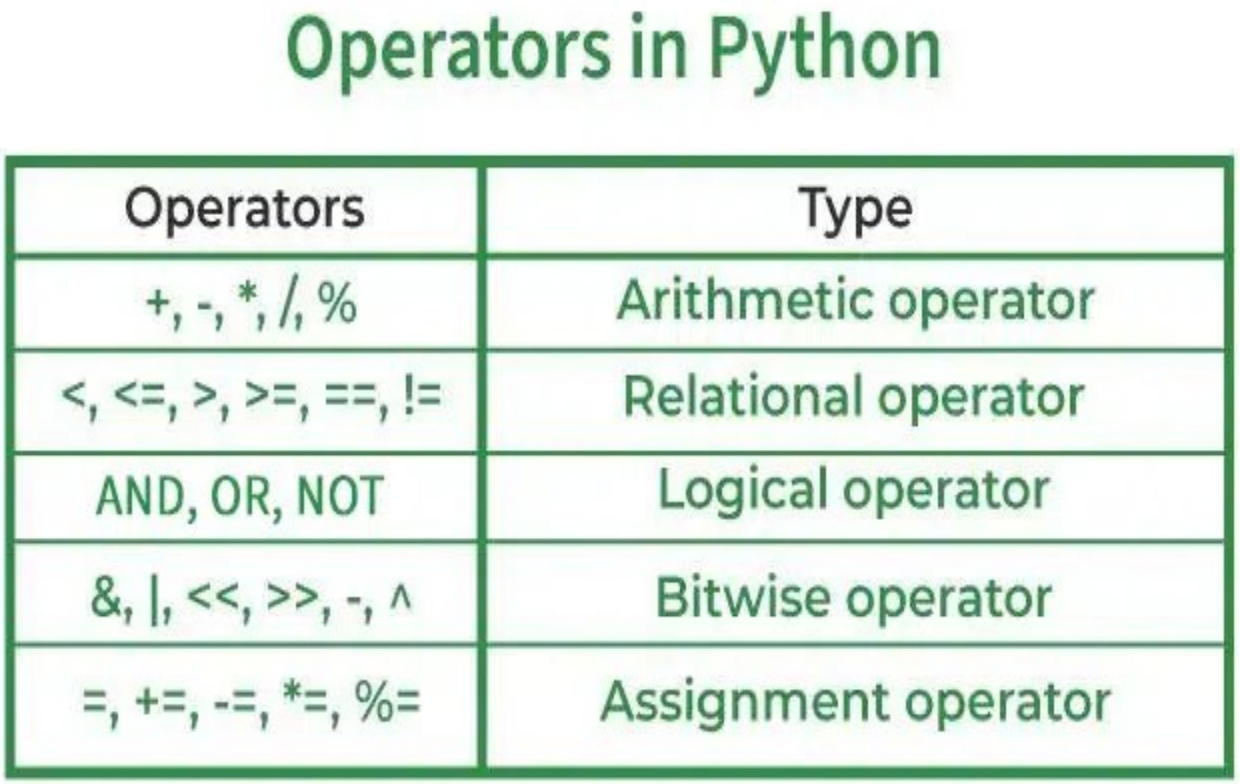
- In order to access items of a dictionary refer to its key name. Key can be used inside square brackets. Using get() method we can access dictionary elements.

```
d = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# Accessing an element using key
print(d['name'])

# Accessing a element using get
print(d.get(3))
```

Types of Operators in Python



Arithmetic Operators

```
# Variables
a = 15
b = 4
# Addition
print("Addition:", a + b)
# Subtraction
print("Subtraction:", a - b)
# Multiplication
print("Multiplication:", a * b)
```

```
# Division
print("Division:", a / b)
# Floor Division
print("Floor Division:", a // b)
# Modulus
print("Modulus:", a % b)
# Exponentiation
print("Exponentiation:", a ** b)
```

Comparison Operators

```
a = 13
b = 33

print(a > b)
print(a < b)
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)
```

Logical Operators

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

Bitwise Operators

Bitwise Operators in Python are as follows:

1. Bitwise NOT
2. Bitwise Shift
3. Bitwise AND
4. Bitwise XOR
5. Bitwise OR

```
a = 10
b = 4

print(a & b)
print(a | b)
print(~a)
print(a ^ b)
print(a >> 2)
print(a << 2)
```

Assignment Operators

```
a = 10
b = a
print(b)
b += a
print(b)
b -= a
print(b)
b *= a
print(b)
b <<= a
print(b)
```

Identity Operators

```
a = 10
b = 20
c = a

print(a is not b)
print(a is c)
```

Membership Operators

```
x = 24
y = 20
list = [10, 20, 30, 40, 50]
if (x not in list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")

if (y in list):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

Ternary Operator

```
a, b = 10, 20
min = a if a < b else b

print(min)
```

Loops and Control Statements

- for Loops

is used to iterate over a sequence (such as a list, tuple, string, or range).

```
# Iterating over a list
a = [1, 2, 3]
for i in a:
    print(i)
```

while Loops

```
# Using while loop
cnt = 0
while cnt < 5:
    print(cnt)
    cnt += 1
```

Control Statements in Loops

- Python provides three primary control statements: continue, break, and pass.

1. break Statement

used to exit the loop prematurely when a certain condition is met.

```
# Using break to exit the loop
for i in range(10):
    if i == 5:
        break
    print(i)
```

- continue Statement

skips the current iteration and proceeds to the next iteration of the loop.

```
# Using continue to skip an iteration
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)
```

- pass Statement

null operation; it does nothing when executed. It's useful as a placeholder for code that you plan to write in the future.

```
# Using pass as a placeholder
for i in range(5):
    if i == 3:
        pass
    print(i)
```